

MI AI API

Version 3.54

©2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.50	Initial release	12/04/2020
3.51	Audio new architecture	07/19/2021
	Added procfs introduction	08/25/2021
3.52	Added MI_AI_GetAttr	09/08/2021
	Modified version: Audio 3.0 Version starts from 3.50(3.0 to 3.49 are Audio 2.0 versions)	12/29/2021
3.53	Added Mochi info	02/17/2022
3.54	Added Maruko info	03/29/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	1
1. 概述.....	3
1.1. 模块说明.....	3
1.2. Audio Codec 框图.....	4
1.2.1 Muffin 系列.....	4
1.2.2 Mochi 系列.....	5
1.2.3 Maruko 系列.....	6
1.3. Audio Codec 器件说明.....	6
1.4. Audio Codec 差异说明.....	7
1.4.1 Muffin 系列.....	7
1.4.2 Mochi 系列.....	8
1.4.3 Maruko 系列.....	8
1.5. API 关键字说明.....	8
2. API 参考.....	12
2.1. MI_AI_Open.....	12
2.2. MI_AI_OpenWithCfgFile.....	15
2.3. MI_AI_Close.....	17
2.4. MI_AI_AttachIf.....	17
2.5. MI_AI_EnableChnGroup.....	18
2.6. MI_AI_DisableChnGroup.....	19
2.7. MI_AI_Read.....	20
2.8. MI_AI_ReleaseData.....	21
2.9. MI_AI_SetGain.....	22
2.10. MI_AI_GetGain.....	22
2.11. MI_AI_SetMute.....	23
2.12. MI_AI_GetMute.....	24
2.13. MI_AI_SetIfGain.....	25
2.14. MI_AI_GetIfGain.....	26
2.15. MI_AI_SetIfMute.....	27
2.16. MI_AI_GetIfMute.....	27
2.17. MI_AI_SetI2SConfig.....	28
2.18. MI_AI_GetI2SConfig.....	29
2.19. MI_AI_DupChnGroup.....	30
2.20. MI_AI_InitDev.....	30
2.21. MI_AI_DeInitDev.....	31
2.22. MI_AI_GetAttr.....	32
3. AI 数据类型.....	33
3.1. MI_AUDIO_DEV.....	33
3.2. MI_AUDIO_Format_e.....	33
3.3. MI_AUDIO_SoundMode_e.....	34
3.4. MI_AUDIO_SampleRate_e.....	35
3.5. MI_AI_Attr_t.....	35

3.6.	MI_AI_If_e.....	36
3.7.	MI_AI_MAX_CHN_NUM.....	38
3.8.	MI_AI_Data_t.....	39
3.9.	MI_AUDIO_I2sMode_e.....	39
3.10.	MI_AUDIO_I2sBitWidth_e.....	40
3.11.	MI_AUDIO_I2sFormat_e.....	40
3.12.	MI_AUDIO_I2sMclk_e.....	41
3.13.	MI_AUDIO_I2sConfig_t.....	42
3.14.	MI_AI_InitParam_t.....	42
4.	错误码.....	44
5.	PROCFS 介绍.....	45
5.1.	cat.....	45
5.2.	echo.....	47

1. 概述

1.1. 模块说明

音频输入（Audio Input, AI）主要实现配置及启用音频输入设备、读取音频数据、以及音量设置等功能。

1.2. Audio Codec 框图

1.2.1 Muffin 系列

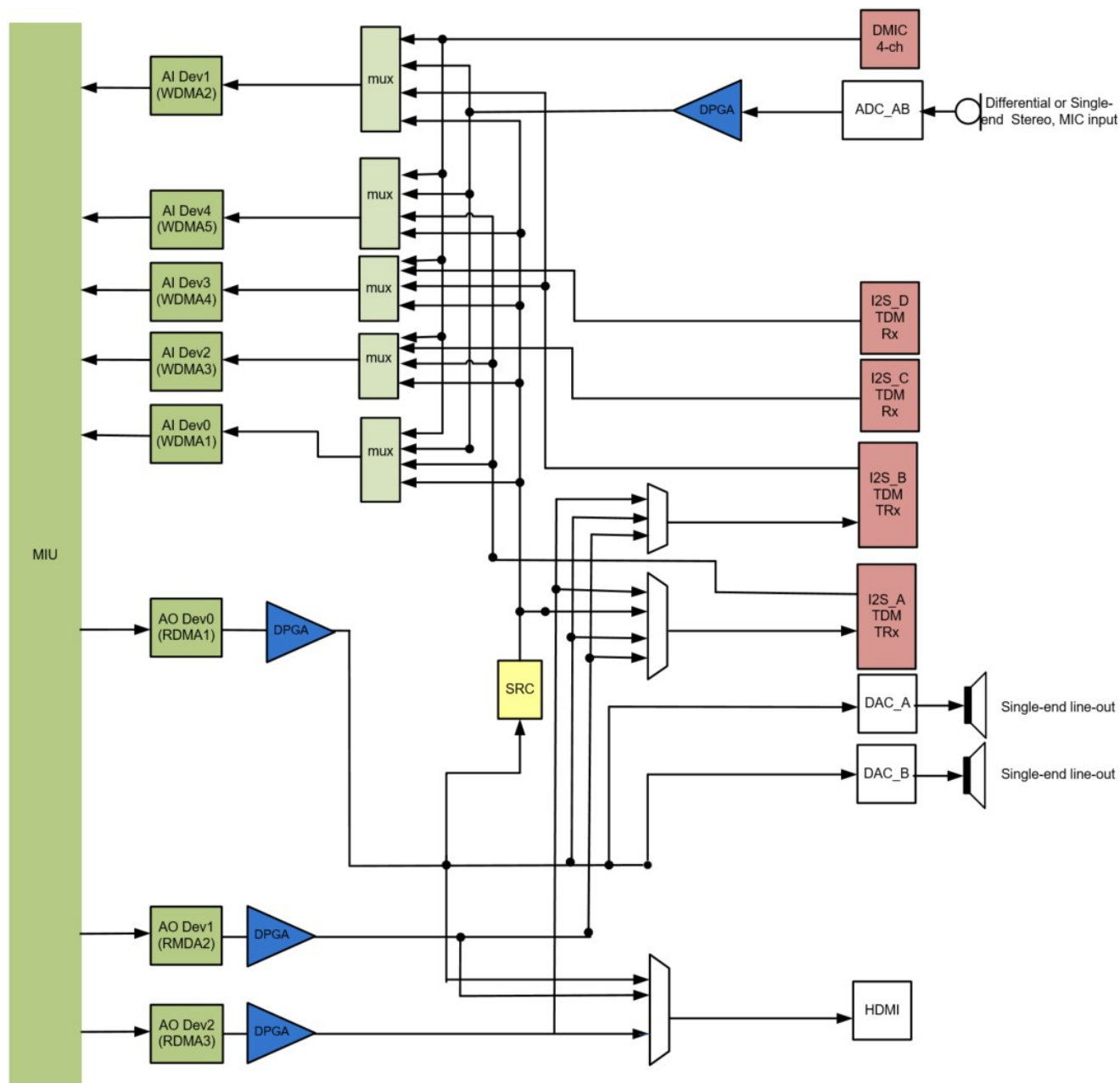


图 1-1: Muffin 系列芯片

Muffin 系列芯片的 audio codec 拥有如下资源: WDMA * 5, RDMA * 3, DMIC 接口(可支持 4Chn DMIC 信号) * 1, ADC(可支持 2Chn Amic/Linein) * 2, I2S-TDM RX * 4, I2S-TDM TX * 2, DAC(可支持 2Chn Lineout) * 2, SRC * 1, HDMI TX * 1。

1.2.2 Mochi 系列

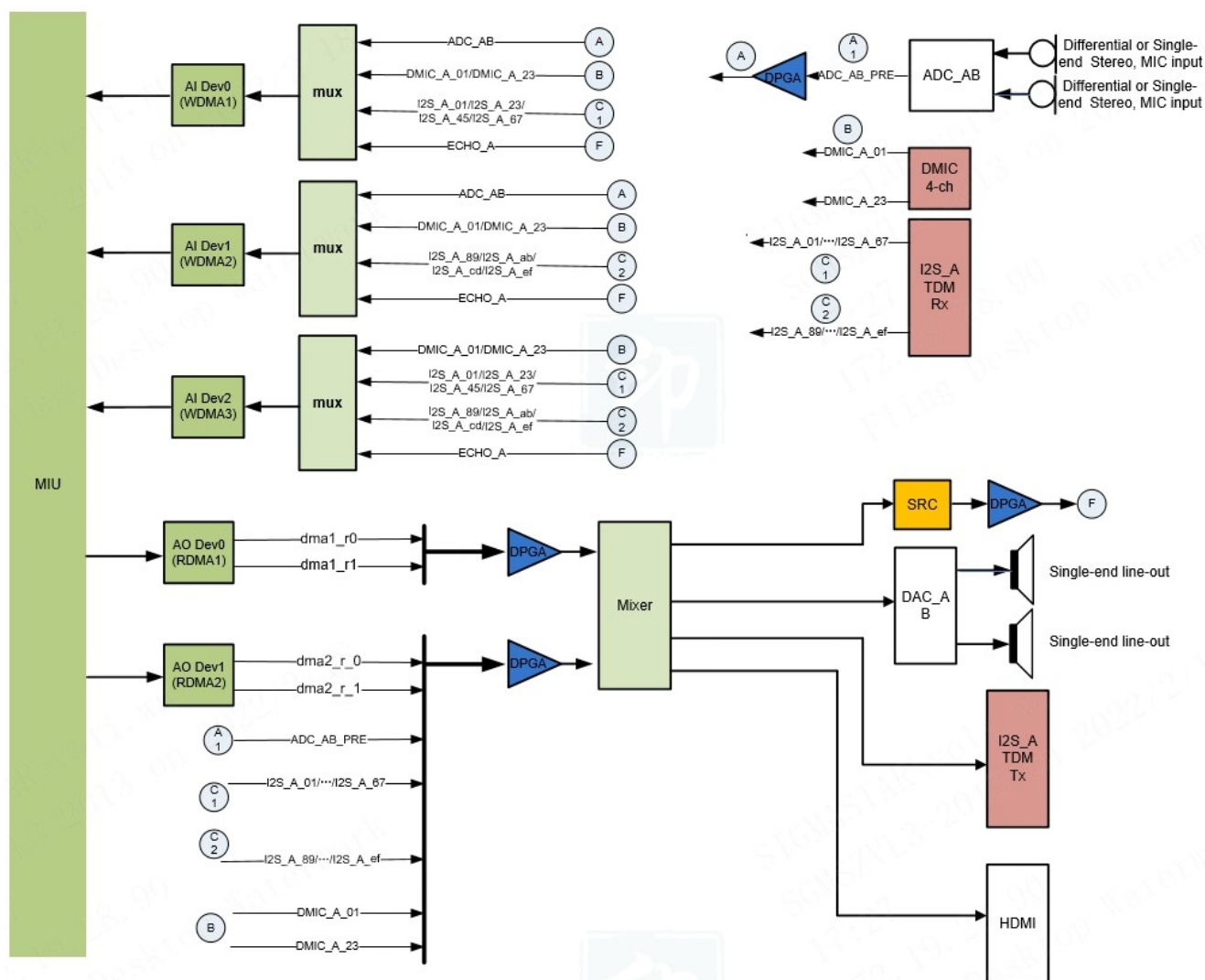


图 1-2: Mochi 系列芯片

Mochi 系列芯片的 audio codec 拥有如下资源: WDMA * 3, RDMA * 2, DMIC 接口(可支持 4Chn DMIC 信号) * 1, ADC(可支持 2Chn Amic/Linein) * 2, I2S-TDM RX(16 slot) * 1, DAC(可支持 2Chn Lineout) * 2, SRC * 1, HDMI TX * 1, I2S-TDM TX(16 slot, 但只有 2 路有效数据) * 1。

1.2.3 Maruko 系列

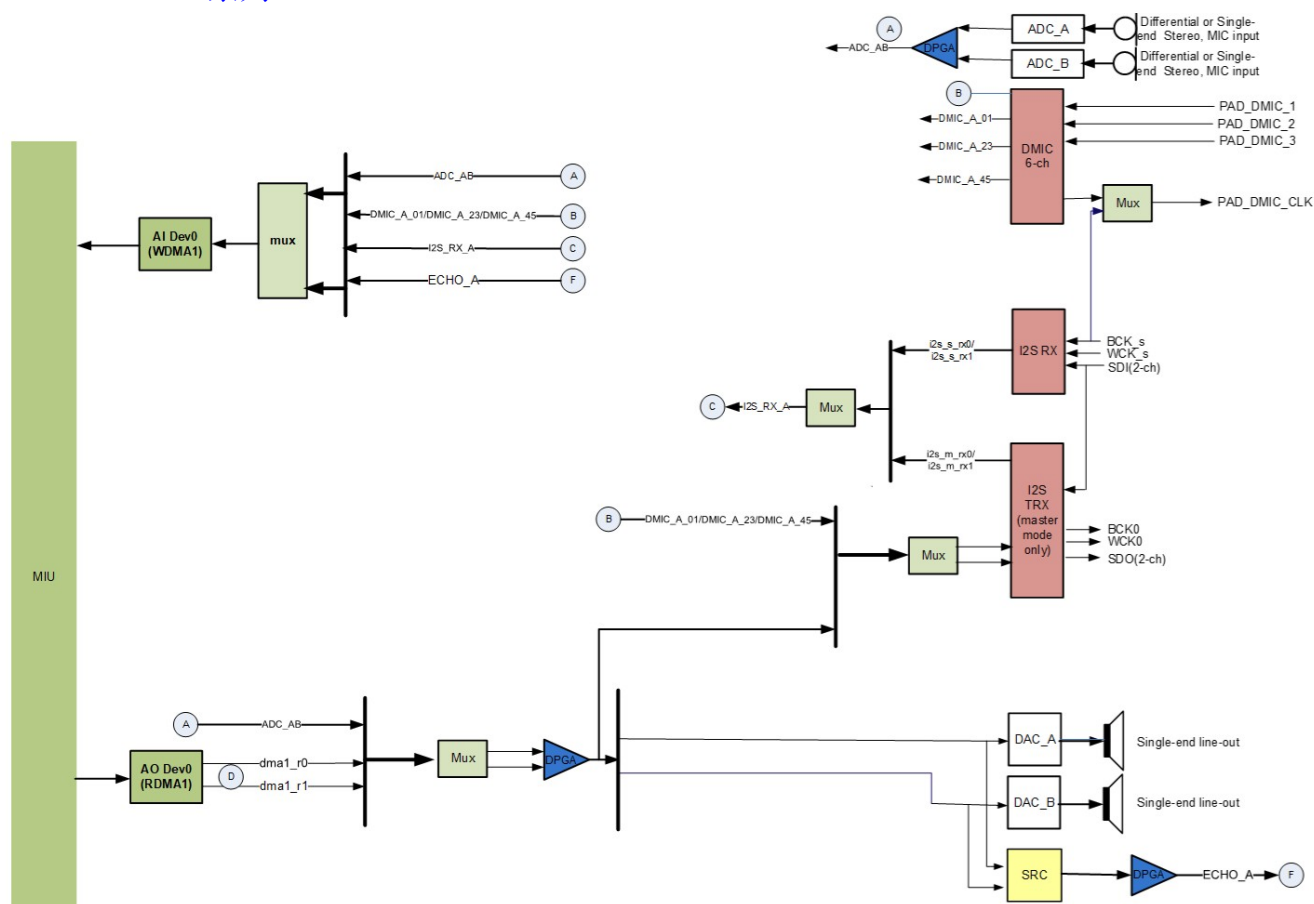


图 1-3: Maruko 系列芯片

Maruko 系列芯片的 audio codec 拥有如下资源: WDMA * 1, RDMA * 1, DMIC 接口(可支持 6Chn DMIC 信号) * 1, ADC(可支持 2Chn Amic/Linein) * 2, I2S RX(2 slot) * 2, DAC(可支持 2Chn Lineout) * 2, SRC * 1, I2S TX(2 slot) * 1。

1.3. Audio Codec 器件说明

● DMA

DMA 即 Direct Memory Access 直接存储器访问, DMA 传输将数据从一个地址空间复制到另一个地址空间, 提供在外设和存储器之间或者存储器和存储器之间的高速数据传输。当 CPU 初始化这个传输动作, 传输动作本身是由 DMA 控制器来实现和完成的。DMA 传输方式无需 CPU 直接控制传输, 也没有中断处理方式那样保留现场和恢复现场过程, 通过硬件为 RAM 和 IO 设备开辟一条直接传输数据的通道, 使得 CPU 的效率大大提高。

● WDMA

WDMA 即 Direct Memory Access Writer 直接存储器访问写入器。

● RDMA

RDMA 即 Direct Memory Access Reader 直接存储器访问读取器。

● MUX

MUX 即 **multiplexer** 数据选择器，在多路数据传送过程中，能够根据需要将其中的任意多路选出来的电路。WDMA 前面的 Mux 为 WDMA 选择多个数据源，可支持选择 1/2/4 个数据源(数据源可以相同也可以不同)，每个数据源各有两声道，即选择 2/4/8 个声道的数据由 WDMA 写到 DRAM（目前最大支持 8 个声道），起到多路开关的作用。而接近输出外设接口(如 I2S TX/HDMI/DAC 等)的 Mux 实现的是多选一的作用。

- **DPGA**
DPGA 即 **Digital Programmable Gain Amplifier** 数字可编程增益放大器，是一种通用性很强的放大器，其放大倍数可以根据需要用程序进行控制。
- **DMIC**
DMIC 即 **Digital Microphone Interface** 数字麦克风接口，audio codec 仅提供 DMIC 接口，并非完整的 DMIC。DMIC 接口提供 DMIC 工作所需要的时钟信号，接收从 DMIC 来的 PDM 信号
- **ADC**
ADC 即 **Analog Digital Conversion** 模拟数字转换，将模拟信号转变为数字信号电子元件。
- **I2S**
I2S 即 **Inter-IC Sound** 集成电路内置音频总线，是飞利浦公司为数字音频设备之间的音频数据传输而制定的一种总线标准。Sigmastar 的 I2S 总线仅支持标准 I2S 数据格式以及左对齐的 I2S 数据格式。同时也支持 TDM 即 **Time-Division Multiplexing** 时分复用技术，将不同的信号相互交织在不同的时间段内，沿着同一个信道传输，可同时支持 4/8/16 通道数据传输。
- **DAC**
DAC 即 **Digital Analog Conversion** 数字模拟转换，将数字信号转变为模拟信号电子元件。
- **SRC**
SRC 即 **Sample Rate Convert** 采样率转换，对语音数字信号进行采样率转换。
- **HDMI_TX**
HDMI(High Definition Multimedia Interface, 高清多媒体接口) TX，是一种全数字化视频和声音发送接口，可以发送未压缩的音频及视频信号。

1.4. Audio Codec 差异说明

1.4.1 Muffin 系列

Muffin 系列芯片拥有 I2S RX * 4，但 I2S RX C 和 I2S RX D 有两种模式，一种称为 **share mode**，一种称为 **slave mode**。share mode，I2S RX C 和 I2S RX A 共享 I2S Clock（Wck 以及 Bck），I2S RX D 和 I2S RX B 共享 I2S Clock（Wck 以及 Bck），当需要同时使用 I2S RX A 和 I2S RX C 时，两者的 I2S 参数必须完全一致，与 I2S RX C 对接的 Codec 需要将 I2S Clock 引脚（Wck 以及 Bck）与 I2S RX A 的 I2S Clock（Wck 以及 Bck）相接，Data 引脚与 I2S RX C 的 Data 相接，硬件解法如下图所示。I2S RX D 与 I2S RX B 的关系也如同 I2S RX C 和 I2S RX A 一般。slave mode，I2S RX C 和 I2S RX D 拥有独立的时钟，但只能作为 **slave** 使用。这两种模式可以通过 **dts sound** 节点下的 **i2s-rx-mode** 来设置，0 为 **slave mode**，1 为 **share mode**。



1.4.2 Mochi 系列

Mochi 系列芯片 I2S 支持最大 16slot，采样率新增支持 96K/192K。注意：在 I2S 的使用场景中，I2S BCK 最大不能超过 30MHz(如：BCK=16bit*16slots*192K=49.152M/ BCK=32bit*16slots*192K=98.304M，BCK > 30MHz 的使用情况不支持)。MUX 目前最大支持 8 个声道，故若需使用 I2S RX 16slot 的情况，需要将 16 声道的数据 attach 到两个 DMA 上。

1.4.3 Maruko 系列

Maruko 系列芯片，虽然有两组 I2S RX，但两组 I2S RX 无法同时使用。以下是两组 I2S RX 的使用场景：

I2S RX:

- 外接 codec 传输的为非 I2S 信号，需要对 Wck、Bck 做调整，比如接收 PCM 信号。
- 需要与 Dmic share clock，以达到 Dmic 和 I2S RX 同步。
- 外接 codec 工作在 master mode。

I2S TRX:

- 需要使用 I2S TX。

两组 I2S RX 选用哪一组通过 `dtb sound` 节点下的 `i2s-pcm` 来选择, 1 为 I2S RX, 0 为 I2S TRX。另外 I2S RX 是否与 `Dmic share clock` 通过 `dmic-bck-share` 来进行配置, 1 为 share clock, 0 为独立 clock。

1.5. API 关键字说明

- Device (音频输入设备)

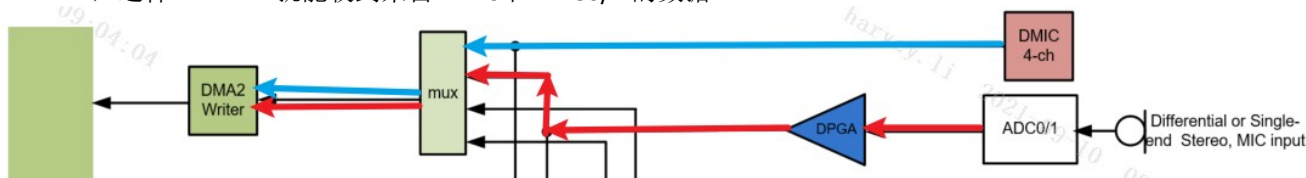
AI 的 Device 指的是 audio codec 的 WDMA。Device 是 audio3.0 对 audio codec 中 DMA 的抽象，audio codec 中的 WDMA 和 AI Device 为一一对应的关系。如 AI Device0 对应 audio codec 中的 WDMA1，AI Device1 对应 audio codec 中的 WDMA2，以此类推。audio3.0 的数据流均以 DMA 为中心进行串接。

- Channel Group (音频输入通道组)

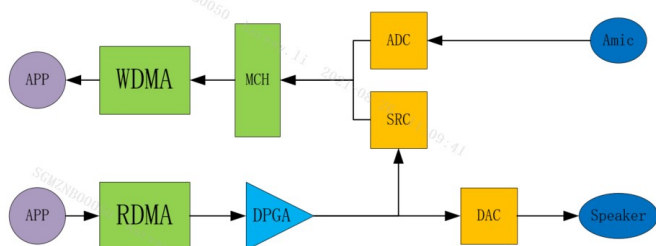
AI 的 Channel Group 指的是多个物理通道组成的集合。而集合具体包含多少个物理声道则由 AI Device 的 Sound Mode 来决定。Channel Group 的意义是为了保证有关联的多个物理声道数据的同步。比如要做一个 4Mic 的 beamforming 算法，4 个物理声道必须保证数据的同步性，解决方法就是把 4 个物理声道当成一个整体来操作。这便是 Channel Group 的意义。而 AI Device 共用多少个 Channel Group，由 AI Device 上所接的物理声道数以及 Sound Mode 共同决定， $\text{Channel Group Number} = (\text{Device 所接的物理声道数} - \text{Device 所接 Echo 的物理声道数}) / \text{Sound Mode}$ 。

- Attach

AI 的 **Attach** 指的是将 **Interface** 挂载到 **WDMA** 对应的 **MUX** 上，即将 **Device** 与 **Interface** 关联起来。对于 **AI Device** 而言，**Attach** 是设定 **WDMA** 对应的 **MUX**，选择哪些 **Interface** 的数据可以通过 **MUX**，由 **WDMA** 做后续的数据搬移。**AI Device** 不支持动态 **Attach Interface**。



AI 的 Echo 指的是 audio codec 提供的 AEC 参考数据。由 audio codec 框图可以看出，SRC 的输入是 RDMA 输出并经过 DPGA 放大的信号，SRC 的输出则是对输入进行重采样后的信号，可通过 Multi Channel 送进 WDMA，作为 AEC 算法的回声参考数据。对于 AI Device 而言，Echo 表示 audio codec 框图中 SRC 的输出信号。而对于 AO Device 而言，Echo 表示将 AO Device 的输出连接到 SRC 的输入。下图以简单框图表示 AI Device 和 AO Device 同时使用 Echo 的数据流情况，应用即可获取到已经对齐的 AEC far end 和 near end 的数据。



AI 的 Sound Mode 表示一个通道组由多少个物理通道组成，而一个 Device 有多少个 Channel Group 取决于 attach 上的物理通道数和 Sound Mode。

- Gain

- **Format**

03/29/2022

- Sample Rate

Sample Rate, 即录音设备在单位时间内对模拟信号采样的次数, 采样频率越高, 机械波的波形越真实越自然。

- Period Size

对于 AI Device 而言, Period Size 表示每个 AI buffer 所包含的数据量(采样样本数)。

- Interleave

对于 AI Device 而言, Interleave 打开与否决定了 MI_AI 如何存放 Channel Group 的数据, 若 Interleave 打开, 则 Channel Group 内的各个物理通道的数据按样本交错存放。若 Interleave 关闭, 则 Channel Group 内的各个物理通道的数据分开存放。

- I2S 参数

1. I2S Mode

I2S Mode 决定了 I2S 的工作模式, 是标准的 I2S 模式还是 Tdm I2S 模式(2Channel 或多 channel), 是 Master 还是 Slave(Master 提供同步时钟, Slave 接收同步时钟)。一般来说, 工作模式没有什么限制, 能与外接的 Codec 时钟匹配上即可。

2. I2S BitWidth

I2S 收发数据的位宽, 目前可支持 16/32bit, 但硬件只能处理 16bit, 意味着位宽为 32bit 时, 低 16bit 为无效数据。

3. I2S Format

I2S 的 Format 即 I2S 的对齐方式, 目前仅支持 I2S Philips 以及 Left-justified 对齐。下图分别为这两种格式的波形图。

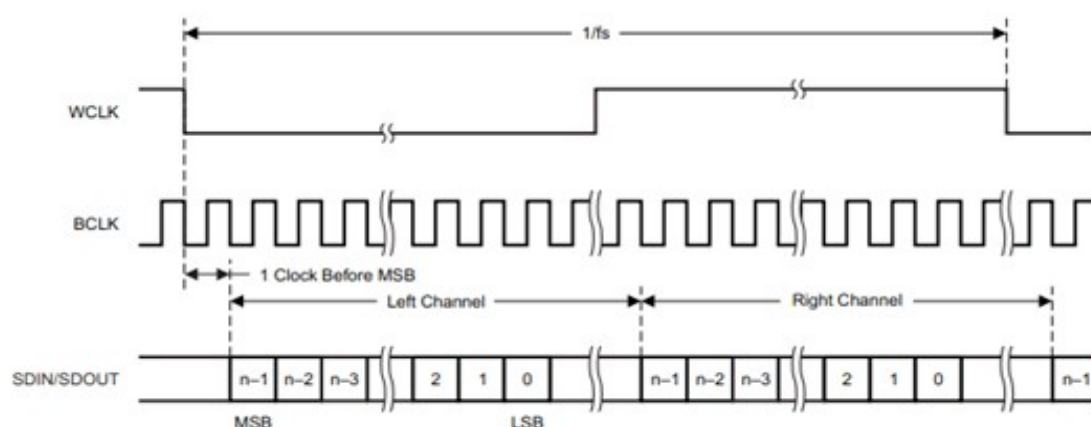
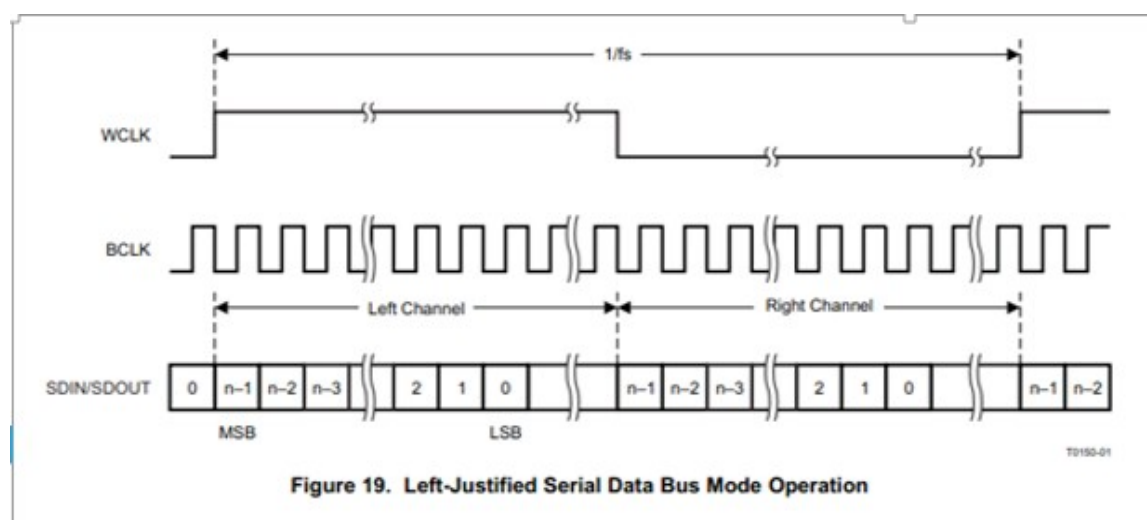


Figure 20. I²S Serial Data Bus Mode Operation



I2S Philips 的对齐格式，样本数据的第一个数据位出现在 WCLK(即左右通道切换时钟)跳变的第一个 BCLK(即串行时钟)后。左对齐格式，样本数据的第一个数据位出现在 WCLK(即左右通道切换时钟)跳变的第一个 BCLK(即串行时钟)内，且 WCLK 的极性与 I2S Philips 的对齐格式相反。

4. I2S Sample Rate

I2S 收发的采样频率。

5. Mclk

Mclk，称为主时钟，也叫系统时钟（System Clock），一般是采样频率的 256 或 384 倍，作用是为了使系统间能够更好的同步，并不是必须的。目前仅支持 12.288M，16.384M，18.432M，24.576M，24M，48M 等。

6. bSyncClock/4-Wire/6-Wire Mode

SigmaStar 的 I2S 有两种接线方式。一种是 4-Wire 模式，包括 RX_WCK、RX_BCK、RX_SDI、TX_SDO 四根线，此模式下，TX 没有独立的 clock，所有的 clock 均由 RX 提供，故在此模式下 TX 需要依赖 RX 来使用，没法单独使用 TX，且 I2S TX 的参数需与 I2S RX 一致。另一种是 6-Wire 模式，包括 RX_WCK、RX_BCK、RX_SDI、TX_WCK、TX_BCK、TX_SDO 六根线，此模式下 RX 和 TX 各自独立，没有关联。

4-Wire/6-Wire Mode 的选择需要根据具体场景来决定。

MI API I2S 参数中的 bSyncClock 为 TRUE，则使用 4-Wire Mode，FALSE 则使用 6-Wire Mode，且属于同一组 I2S 的 RX 和 TX 不能一边设置成 4-Wire Mode，另一边设置成 6-Wire Mode。

7. Slot

Slot 表示 I2S 传输的声道数，目前 I2S 模式下支持 2slot，Tdm 模式下支持 4/8slot (Mochi 系列支持 16slot)。

2. API 参考

音频输入（AI）主要实现音频输入设备的配置和使能、获取音频帧数据等功能。

API 名	功能
MI_AI_Open	打开音频输入设备
MI_AI_OpenWithCfgFile	打开音频输入设备，并按配置文件初始化
MI_AI_Close	关闭音频输入设备
MI_AI_AttachIf	挂载外设到音频输入设备
MI_AI_EnableChnGroup	使能音频输入通道组
MI_AI_DisableChnGroup	禁用音频输入通道组
MI_AI_Read	读取音频数据
MI_AI_ReleaseData	释放音频数据
MI_AI_SetGain	设置音频输入通道组的音量
MI_AI_GetGain	获取音频输入通道组的音量
MI_AI_SetMute	设置音频输入通道组的静音参数
MI_AI_GetMute	获取音频输入通道组的静音参数
MI_AI_SetIfGain	设置音频输入外设的音量
MI_AI_GetIfGain	获取音频输入外设的音量
MI_AI_SetIfMute	设置音频输入外设的静音参数
MI_AI_GetIfMute	获取音频输入外设的静音参数
MI_AI_SetI2SConfig	设置 I2S RX 的配置信息
MI_AI_GetI2SConfig	获取 I2S RX 的配置信息
MI_AI_DupChnGroup	同步音频输入通道组状态
MI_AI_InitDev	初始化音频输入设备
MI_AI_DeInitDev	反初始化音频输入设备
MI_AI_GetAttr	获取音频输入设备属性

2.1. MI_AI_Open

➤ 功能

打开音频输入设备。

➤ 语法

```
MI_S32 MI_AI_Open(MI_AUDIO_DEV AiDevId, const MI_AI_Attr_t *pstAttr);
```

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
pstAttr	设备属性指针	输入

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so`

※ 注意

音频输入设备属性包括数据格式、声音模式、采样率、音频数据的样本数、交错模式。

- 音频数据格式([MI_AUDIO_Format_e](#))
音频数据格式指采样样本的数据格式。目前仅支持 `S16_LE`。
- 音频声音模式([MI_AUDIO_SoundMode_e](#))
音频声音模式指通道组包含的物理通道数。`MONO` 表示 1 个通道组包含 1 个物理通道，`STEREO` 表示 1 个通道组包含 2 个物理通道，`4CH` 表示 1 个通道组包含 4 个物理通道，`6CH` 表示 1 个通道组包含 6 个物理通道，`8CH` 表示 1 个通道组包含 8 个物理通道。
- 音频采样率([MI_AUDIO_SampleRate_e](#))
音频采样率指一秒内采样的次数。采样率越高，采样的结果与原信号相比失真度越小，但处理的数据量也就随之增加。
- 音频数据的样本数(`u32PeriodSize`)
音频数据的样本数即每次读取音频数据包含的样本个数。当音频采样率较高时，建议相应地增大 `u32PeriodSize`。若发现采集到的声音断续，则需要增大 `u32PeriodSize` 和缓存 `buffer` 大小。
- 交错模式(`bInterleaved`)
交错模式是否使能决定了通道组中各物理通道的数据排列情况。以声音模式为 `4CH` 举例，下面以 `Sx` 代替第 `x` 个 `sample` 进行描述，当交错模式使能时，4 个物理通道的数据交错排列，如 `Chn0S0 Chn1S0 Chn2S0 Chn3S0 Chn0S1 Chn1S1 Chn2S1 Chn3S1.....`，当交错模式禁用时，4 个物理通道的数据按通道顺序，一个通道排列完再接下一个通道，如
`Chn0S0 Chn0S1 Chn0S2 Chn0S3 ..... Chn0Sn`
`Chn1S0 Chn1S1 Chn1S2 Chn1S3 ..... Chn1Sn`
`Chn2S0 Chn2S1 Chn2S2 Chn2S3 ..... Chn2Sn`
`Chn3S0 Chn3S1 Chn3S2 Chn3S3 ..... Chn3Sn`

➤ 举例

简单例子如下：

```
1. MI_AI_Attr_t stAiSetAttr = {0};
2. MI_AUDIO_DEV AiDevId = 0;
3. stAiSetAttr.enFormat = E_MI_AUDIO_FORMAT_PCM_S16_LE;
4. stAiSetAttr.enSoundMode = E_MI_AUDIO_SOUND_MODE_MONO;
5. stAiSetAttr.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
6. stAiSetAttr.u32PeriodSize = 1024;
7. stAiSetAttr.bInterleaved = TRUE;
8. ExecFunc(MI_AI_Open(AiDevId, &stAiSetAttr), MI_SUCCESS);
```


详细例子如下:

```
1. MI_AI_Attr_t stAiSetAttr = {0};
2. MI_AI_Attr_t stAiGetAttr = {0};
3. MI_AUDIO_DEV AiDevId = 0;
4. MI_AI_If_e enAiIf[] = {E_MI_AI_IF_ADC_AB};
5. MI_AI_Data_t stAiChFrame;
6. MI_AI_Data_t stAecFrame;
7. MI_U8 u8ChnGrpId = 0;
8. MI_SYS_ChnPort_t stAiChnOutputPort;
9. MI_S8 s8DpgaGain[] = {-10};
10.
11. // 设置AI Device 的 Format 为 S16_LE
12. stAiSetAttr.enFormat = E_MI_AUDIO_FORMAT_PCM_S16_LE;
13.
14. // 设置 Sound Mode 为 Mono, 一个 Channel Group 对应一个物理声道
15. stAiSetAttr.enSoundMode = E_MI_AUDIO_SOUND_MODE_MONO;
16.
17. // 设置AI Device 的采样率为 8KHz
18. stAiSetAttr.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
19.
20. // 设置每个AI Buffer 包含 1024 个采样样本
21. stAiSetAttr.u32PeriodSize = 1024;
22.
23. // 设置AI buffer 的数据排列形式
24. stAiSetAttr.bInterleaved = TRUE;
25.
26. // 打开AI Device
27. ExecFunc(MI_AI_Open(AiDevId, &stAiSetAttr), MI_SUCCESS);
28.
29. // 获取设备属性
30. ExecFunc(MI_AI_GetAttr(AiDevId, &stAiGetAttr), MI_SUCCESS);
31.
32. // 将ADC0/1 Attach 到WDMA1 上
33. ExecFunc(MI_AI_AttachIf(AiDevId, enAiIf, sizeof(enAiIf) / sizeof(enAiIf[0])), MI_SUCCESS);
34.
35. // 设置 output depth
36. memset(&stAiChnOutputPort, 0, sizeof(stAiChnOutputPort));
37. stAiChnOutputPort.eModId = E_MI_MODULE_ID_AI;
38. stAiChnOutputPort.u32DevId = AiDevId;
39. stAiChnOutputPort.u32ChnId = u8ChnGrpId;
40. stAiChnOutputPort.u32PortId = 0;
41. ExecFunc(MI_SYS_SetChnOutputPortDepth(0, &stAiChnOutputPort, 4, 8), MI_SUCCESS);
42.
43. // 设置ADC0/1 的Interface gain
44. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_ADC_AB, 18, 0), MI_SUCCESS);
45.
46. // 设置ADC0/1 的DPGA gain
47. ExecFunc(MI_AI_SetGain(AiDevId, u8ChnGrpId, s8DpgaGain, sizeof(s8DpgaGain) / sizeof(s8DpgaGain[0])), MI_SUCCESS);
48.
49.
50. // 使能 Channel Group
51. ExecFunc(MI_AI_EnableChnGroup(AiDevId, u8ChnGrpId), MI_SUCCESS);
52.
53. // 获取音频帧数据
54. MI_AI_Read(AiDevId, u8ChnGrpId, &stAiChFrame, &stAecFrame, -1);
55.
56. // do something
```

```

57.
58. // 释放音频帧数据
59. MI_AI_ReleaseData(AiDevId, u8ChnGrpId, &stAiChFrame, &stAecFrame);
60.
61. // 禁用 Channel Group
62. ExecFunc(MI_AI_DisableChnGroup(AiDevId, u8ChnGrpId), MI_SUCCESS);
63.
64. // 关闭 AI Device
65. ExecFunc(MI_AI_Close(MI_AI_DEV_1), MI_SUCCESS);

```

2.2. MI_AI_OpenWithCfgFile

➤ 功能

打开音频输入设备，并按配置文件初始化。

➤ 语法

MI_S32 MI_AI_OpenWithCfgFile([MI_AUDIO_DEV](#) AiDevId, const char *pCfgPath);

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
pCfgPath	配置文件路径	输入

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so

※ 注意

- 此接口相当于 [MI_AI_Open](#) 和 [MI_AI_AttachIf](#) 组合。
- 如果音频输入设备已经处于开启状态，则直接返回成功。
- 配置文件模板如下：

```

1. ; Device attr section
2. [DEV]
3. ; enFormat determines the data format of the sample, only supports S16_LE now.
4. ; 0[S16_LE]
5. enFormat = 0
6. ; enSoundMode determines the channel count of Channel Group.
7. ; 1[Mono] 2[Stereo] 4[4Chn] 6[6Chn] 8[8Chn]
8. enSoundMode = 2
9. ; enSampleRate determines the sample rate of AI Device.
10. ; 8000[8KHz] 16000[16KHz] 32000[32KHz] 48000[48KHz]
11. enSampleRate = 8000
12. ; u32PeriodSize determines the count of sample in a AI Buffer
13. u32PeriodSize = 1024
14. ; bInterleaved determines the data arrangement mode of each channel.
15. ; if bInterleaved = 1, Data on each channel is interleaved, Otherwise, data on each channel
   is stored separately.
16. bInterleaved = 0

```

```

17. ; attach interface
18. ; 1[ADC_AB] 2[ADC_CD] 3[DMIC_A_01] 4[DMIC_A_23]
19. ; 5[I2S_A_01] 6[I2S_A_23] 7[I2S_A_45] 8[I2S_A_67] 9[I2S_A_89] 10[I2S_A_ab] 11[I2S_A_cd] 12[I2S_A_ef]
20. ; 13[I2S_B_01] 14[I2S_B_23] 15[I2S_B_45] 16[I2S_B_67] 17[I2S_B_89] 18[I2S_B_ab] 19[I2S_B_cd] 20[I2S_B_ef]
21. ; 21[I2S_C_01] 22[I2S_C_23] 23[I2S_C_45] 24[I2S_C_67] 25[I2S_C_89] 26[I2S_C_ab] 27[I2S_C_cd] 28[I2S_C_ef]
22. ; 29[I2S_D_01] 30[I2S_D_23] 31[I2S_D_45] 32[I2S_D_67] 33[I2S_D_89] 34[I2S_D_ab] 35[I2S_D_cd] 36[I2S_D_ef]
23. ; 37[ECHO_A] 38[HDMI_A]
24. aenAiIfs={1,5}
25.
26. ; I2S RX attr section
27. ; It is not necessary when you're not using I2S RX
28. [I2S_A]
29. ; enMode determines the working mode of I2S Rx
30. ; 0[I2S Master] 1[I2S Slave] 2[Tdm Master] 3[Tdm Slave]
31. enMode = 0
32. ; enBitWidth determines the bit width of I2S Rx
33. ; 0[16 bit] 1[32bit]
34. enBitWidth = 0
35. ; enFormat determines the waveform alignment of I2S Rx
36. ; 0[I2S Philips] 1[I2S Left-justify]
37. enFormat = 0
38. ; enSampleRate determines the sample rate of I2S Rx
39. ; 8000[8KHz] 16000[16KHz] 32000[32KHz] 48000[48KHz]
40. enSampleRate = 8000
41. ; enMclk determines the frequency of Mclk
42. ; 0[disable Mclk] 1[12.288M] 2[16.384M] 3[18.432M] 4[24.576M] 5[24M] 6[48M]
43. enMclk = 0
44. ; bSyncClock: 0[4-wire mode] 1[6-wire mode]
45. bSyncClock = 0
46. ; u32TdmSlots determines the slot number of I2S Rx
47. u32TdmSlots = 2

```

配置项的取值与下给 MI API 的参数一致，enFormat = 0 表示 AI Device 采用 S16_LE 格式，enSoundMode = 1 表示 Sound Mode 采用 Mono，enSampleRate = 8000 表示 AI Device 使用 8KHz 采样频率，u32PeriodSize = 1024 表示一个 AI Buffer 包含 1024 个采样点，bInterleaved = 1 表示 Channel Group 的数据排列按交错方式排列，aenAiIfs={1}，表示将 ADC0/1 attach 到 WDMA。若需要使用 I2S RX 还需要设置 I2S RX 的参数，上述配置作用于 I2S_A，enMode = 0 表示使用 I2S Master 模式，enBitWidth = 0 表示 I2S 的接收位宽为 16bit，enFormat = 0 表示按 I2S Philips 方式对齐，enSampleRate = 8000 表示 I2S RX 的采样频率为 8KHz，enMclk = 0 表示不使用 Mclk，bSyncClock = 0 表示 4-wire 模式，u32TdmSlots 表示接收 2 通道数据。注意使用时需把注释行删掉。



AI template.ini

➤ 举例

```

1. char *path = "/tmp/Dev0Cfg.ini";
2. ExecFunc(MI_AI_OpenWithCfgFile(AiDevId, path), MI_SUCCESS);

```

2.3. MI_AI_Close

➤ 功能

关闭音频输入设备。

➤ 语法

```
MI_S32 MI_AI_Close(MI\_AUDIO\_DEV AiDevId);
```

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so`

※ 注意

- 如果音频输入设备已经处于关闭状态，则直接返回成功。

➤ 举例

简单例子如下：

```
1. ExecFunc(MI_AI_Close(MI_AI_DEV_1), MI_SUCCESS);
```

详细例子请参考 [MI_AI_Open](#)。

2.4. MI_AI_AttachIf

➤ 功能

挂载外设到音频输入设备。

➤ 语法

```
MI_S32 MI_AI_AttachIf(MI\_AUDIO\_DEV AiDevId, const MI\_AI\_If\_e aenAiIfs[], MI_U8 u8AiIfSize);
```

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
aenAiIfs	AI Interface数组，需要挂载到音频输入设备的外设信息；数组的一个元素表示2个相邻物理通道所挂载的外设	输入
U8AiIfSize	interface数组的大小，最大为(MI_AI_MAX_CHN_NUM /2)	输入

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so`

※ 注意

- 此接口仅能在 [MI_AI_Open](#) 成功后调用。
- 如需要挂载 I2S RX 到音频输入设备，需要先调用 [MI_AI_SetI2SConfig](#) 来初始化 I2S RX。
- 如果要获取回声参考数据，AI 模块需要 attach `E_MI_AI_IF_ECHO_A`，AO 模块需要 attach `E_MI_AO_IF_ECHO_A`。在 AI 模块中 `E_MI_AI_IF_ECHO_A` 必须与非 `E_MI_AI_IF_ECHO_A` 的 interface 一同 attach，且必须放在非 `E_MI_AI_IF_ECHO_A` 的 interface 后面。
- 当 attach 的 interface 通道数（减去 `E_MI_AI_IF_ECHO_A` 的通道数）少于或不能整除于音频声音模式 ([MI_AUDIO_SoundMode_e](#)) 时，返回错误。

➤ 举例

简单例子如下：

```
1. MI_AI_If_e enAiIf[] = {E_MI_AI_IF_ADC_AB, E_MI_AI_IF_ADC_CD};
2. ExecFunc(MI_AI_AttachIf(AiDevId, enAiIf, sizeof(enAiIf) / sizeof(enAiIf[0])), MI_SUCCESS);
```

详细例子请参考 [MI_AI_Open](#)。

2.5. MI_AI_EnableChnGroup

➤ 功能

使能音频输入通道组。

➤ 语法

`MI_S32 MI_AI_EnableChnGroup(MI\_AUDIO\_DEV AiDevId, MI_U8 u8ChnGrpIdx);`

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 音频输入通道组总数 = (音频输入设备挂载外设的物理通道数 - 回声所占用的物理通道数) / 音频输入设备的声音模式 音频输入通道组号的范围为[0, 音频输入通道组总数 - 1]	输入

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so`

※ 注意

- 此接口仅能在 [MI_AI_Open](#) 和 [MI_AI_AttachIf](#) 成功后调用。
- 如果音频输入通道组已使能，则直接返回成功。
- 音频输入通道组的操作不支持多进程，在哪个进程使能，只能在哪个进程使用，且只能在该进程禁用。

➤ 举例

简单例子如下：

1. ExecFunc(MI_AI_EnableChnGroup(AiDevId, 0), MI_SUCCESS);

详细例子请参考 [MI_AI_Open](#)。

2.6. MI_AI_DisableChnGroup

➤ 功能

禁用音频输入通道组。

➤ 语法

MI_S32 MI_AI_DisableChnGroup([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx);

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 范围为[0, Channel Group Number - 1] Channel Group Number = (attach上的物理通道数 - attach上的Echo通道数) / Sound Mode	输入

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非0} & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件: [mi_ai.h](#)
- 库文件: [libmi_ai.a/libmi_ai.so](#)

※ 注意

- 如果 AI 通道已禁用，则直接返回成功。
- 音频输入通道组的操作不支持多进程，在哪个进程使能，只能在哪个进程使用，且只能在该进程禁用。

➤ 举例

简单例子如下：

1. ExecFunc(MI_AI_DisableChnGroup(AiDevId, 0), MI_SUCCESS);

详细例子请参考 [MI_AI_Open](#)。

2.7. MI_AI_Read

➤ 功能

读取音频数据。

➤ 语法

MI_S32 MI_AI_Read([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx, [MI_AI_Data_t](#) *pstData, [MI_AI_Data_e](#) *pstEchoRefData, MI_S32 s32TimeoutMs);

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 范围为[0, Channel Group Number - 1] Channel Group Number = (attach上的物理通道数 - attach上的Echo通道数) / Sound Mode	输入
pstData	音频数据结构体指针	输出
pstEchoRefData	回声参考数据结构体指针	输出
s32TimeoutMs	获取数据的超时时间 -1 表示阻塞模式，无数据时一直等待 0 表示非阻塞模式，无数据时则报错返回 >0 表示阻塞s32TimeoutMs毫秒，超时则报错返回	输入

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so`

※ 注意

- 此接口仅能在 [MI_AI_Open](#) / [MI_AI_AttachIf](#) / [MI_AI_EnableChnGroup](#) 成功后调用。
- 如果需要获取回声参考数据，`pstEchoRefData` 不能是空指针，如果不想获取回声参考数据 `pstEchoRefData` 置为空指针即可。
- `s32TimeoutMs` 的值必须大于等于-1，等于-1 时采用阻塞模式获取数据，等于 0 时采用非阻塞模式获取数据，大于 0 时，阻塞 `s32TimeoutMs` 毫秒后，没有数据则返回超时并报错。
- 本接口支持 `select/poll` 操作，建议使用 `select/poll` 操作代替超时操作。

➤ 举例

简单例子如下：

```
1. MI_AI_Data_t stAiChFrame;
2. MI_AI_Data_t stAecFrame;
3. MI_AI_Read(AiDevId, AiChnGroup, &stAiChFrame, &stAecFrame, -1);
```

详细例子请参考 [MI_AI_Open](#)。

2.8. MI_AI_ReleaseData

➤ 功能

释放音频数据。

➤ 语法

```
MI_S32 MI_AI_ReleaseData(MI_AUDIO_DEV AiDevId, MI_U8 u8ChnGrpIdx, MI_AI_Data_t *pstData, MI_AI_Data_t *pstEchoRefData);
```

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 范围为[0, Channel Group Number - 1] Channel Group Number = (attach上的物理通道数 - attach上的Echo通道数) / Sound Mode	输入
pstData	音频数据结构体指针	输入
pstEchoRefData	回声参考数据结构体指针	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so

※ 注意

无

➤ 举例

简单例子如下：

```
1. MI_AI_ReleaseData(AiDevId, AiChnGroup, &stAiChFrame, &stAecFrame);
```

详细例子请参考 [MI_AI_Open](#)。

2.9. MI_AI_SetGain

➤ 功能

设置音频输入通道组的音量。

➤ 语法

```
MI_S32 MI_AI_SetGain(MI\_AUDIO\_DEV AiDevId, MI_U8 u8ChnGrpIdx, const MI_S8 as8Gains[], MI_U8 u8GainSize);
```

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 范围为[0, Channel Group Number - 1] Channel Group Number = (attach上的物理通道数 - attach上的Echo通道数) / Sound Mode	输入
as8Gains	音量数组 数组的一个元素表示一个物理通道的音量 范围为[-60, 30]	输入
u8GainSize	音量数组的大小，最大为Sound Mode所对应的物理通道	输入

➤ 返回值

返回值 { 0 成功。

非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so`

※ 注意

- 设置音频输入通道组的音量，必须先打开音频输入设备且挂载上外设。
- Muffin 系列芯片仅 `E_MI_AI_IF_ADC_AB` 对应的通道支持此设定，其他 `interface` 无效。

➤ 举例

简单例子如下：

```
1. MI_S8 s8DpgaGain[] = {-10};
2. ExecFunc(MI_AI_SetGain(AiDevId, u8ChnGroupIdx, s8DpgaGain, sizeof(s8DpgaGain) / sizeof(s8DpgaGain[0])), MI_SUCCESS);
```

详细例子请参考 [MI_AI_Open](#)。

2.10. MI_AI_GetGain

➤ 功能

获取音频输入通道组的音量。

➤ 语法

MI_S32 MI_AI_GetGain([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx, MI_S8 as8Gains[], MI_U8 *pu8GainSize);

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 范围为[0, Channel Group Number - 1] Channel Group Number = (attach上的物理通道数 - attach上的Echo通道数) / Sound Mode	输入
as8Gains	音量数组 数组的一个元素表示一个物理通道的音量	输出
pu8GainSize	音量数组的大小，最大为Sound Mode所对应的物理通道	输出

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so`

※ 注意

- 获取音频输入通道组的音量，必须先打开音频输入设备且挂载上外设。
- Muffin 系列芯片仅 `E_MI_AI_IF_ADC_AB` 对应的通道支持此接口，其他 `interface` 无效。

➤ 举例

简单例子如下：

```
1. MI_S8 s8DpgaGain[MI_AI_MAX_CHN_NUM];
2. MI_U8 u8Size;
3. memset(&s8DpgaGain, 0x0, sizeof(s8DpgaGain));
4. ExecFunc(MI_AI_GetGain(AiDevId, u8ChnGroupIdx, s8DpgaGain, &u8Size), MI_SUCCESS);
```

详细例子请参考 [MI_AI_Open](#)。

2.11. MI_AI_SetMute

➤ 功能

设置音频输入通道组的静音参数。

➤ 语法

```
MI_S32 MI_AI_SetMute(MI\_AUDIO\_DEV AiDevId, MI_U8 u8ChnGrpIdx, const MI_BOOL abMutes[],
MI_U8 u8MuteSize);
```

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 范围为[0, Channel Group Number - 1] Channel Group Number = (attach上的物理通道数 - attach上的Echo通道数) / Sound Mode	输入
abMutes	静音参数数组 静音参数数组的一个元素表示一个物理通道的静音参数	输入
u8MuteSize	静音参数数组大小, 最大为Sound Mode所对应的物理通道数	输入

➤ 返回值

返回值

{	0	成功。
	非 0	失败, 参照 错误码 。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so`

※ 注意

- 设置音频输入通道组的静音参数, 必须先打开音频输入设备且挂载上外设。

➤ 举例

```
1. MI_BOOL bDpgaMute[] = {TRUE, FALSE};
2. ExecFunc(MI_AI_SetMute(AiDevId, u8ChnGroupIdx, bDpgaMute, sizeof(bDpgaMute) / sizeof(bDpgaMute[0])), MI_SUCCESS);
```

2.12. MI_AI_GetMute

➤ 功能

获取音频输入通道组的静音参数。

➤ 语法

```
MI_S32 MI_AI_GetMute(MI_AUDIO_DEV AiDevId, MI_U8 u8ChnGrpIdx, MI_BOOL abMutes[], MI_U8 *pu8MuteSize);
```

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 范围为[0, Channel Group Number - 1] Channel Group Number = (attach上的物理通道数 - attach上的Echo通道数) / Sound Mode	输入
abMutes	静音参数数组 静音参数数组的一个元素表示一个物理通道的静音参数	输出
pu8MuteSize	静音参数数组大小, 最大为最大为Sound Mode所对应的物理通道	输出

➤ 返回值

返回值 $\left\{ \begin{array}{ll} 0 & \text{成功。} \\ \text{非0} & \text{失败, 参照[错误码](#)。} \end{array} \right.$

➤ 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so

※ 注意

- 获取音频输入通道组的静音参数, 必须先打开音频输入设备且挂载上外设。

➤ 举例

```
1. MI_BOOL bDpgaMute[MI_AI_MAX_CHN_NUM];
2. MI_U8 u8Size;
3. memset(&bDpgaMute, 0x0, sizeof(bDpgaMute));
4. ExecFunc(MI_AI_GetMute(AiDevId, u8ChnGroupIdx, s8DpgaGain, &u8Size), MI_SUCCESS);
```

2.13. MI_AI_SetIfGain

➤ 功能

设置音频输入外设的音量。

➤ 语法

```
MI_S32 MI_AI_SetIfGain(MI_AI_If_e enAiIf, MI_S8 s8LeftIfGain, MI_S8 s8RightIfGain);
```

➤ 形参

参数名称	描述	输入/输出
enAiIf	音频输入外设	输入
s8LeftIfGain	左声道音量	输入
s8RightIfGain	右声道音量	输入

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so

※ 注意

- 目前仅 E_MI_AI_IF_ADC_AB/E_MI_AI_IF_ADC_CD/E_MI_AI_IF_DMIC_A_01/E_MI_AI_IF_DMIC_A_23/ E_MI_AI_IF_DMIC_A_45 支持此设定。
- E_MI_AI_IF_ADC_AB/E_MI_AI_IF_ADC_CD 支持的音量范围: Muffin/Mochi[0, 19], 对应到 0~57Db, 3dB/step; Maruko[0, 21], 对应到 -6~57Db, 3dB/step。
- E_MI_AI_IF_DMIC_A_01/E_MI_AI_IF_DMIC_A_23/ E_MI_AI_IF_DMIC_A_45 支持的音量范围为[0,6], 对应到 0~36dB, 6dB/step。

➤ 举例

简单例子如下:

```
1. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_ADC_AB, 18, 18), MI_SUCCESS);
2. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_ADC_CD, 18, 18), MI_SUCCESS);
3. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_DMIC_A_01, 4, 4), MI_SUCCESS);
4. ExecFunc(MI_AI_SetIfGain(E_MI_AI_IF_DMIC_A_23, 4, 4), MI_SUCCESS);
```

详细例子请参考 [MI_AI_Open](#)。

2.14. MI_AI_GetIfGain

➤ 功能

获取音频输入外设的音量。

➤ 语法

```
MI_S32 MI_AI_GetIfGain(MI\_AI\_If\_e enAiIf, MI_S8 *ps8LeftIfGain, MI_S8 *ps8RightIfGain);
```

➤ 形参

参数名称	描述	输入/输出
enAiIf	音频输入外设	输入
ps8LeftIfGain	左声道音量指针	输出
ps8RightIfGain	右声道音量指针	输出

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.a/libmi_ai.so`
- ※ 注意
 - 目前仅 `E_MI_AI_IF_ADC_AB/E_MI_AI_IF_ADC_CD/E_MI_AI_IF_DMIC_A_01/E_MI_AI_IF_DMIC_A_23/ E_MI_AI_IF_DMIC_A_45` 支持此设定。

- 举例


```
1. MI_S8 s8IfLeftGain, s8IfRightGain;
2. ExecFunc(MI_AI_GetIfGain(E_MI_AI_IF_ADC_AB, &s8IfLeftGain, &s8IfRightGain), MI_SUCCESS);
3. ExecFunc(MI_AI_GetIfGain(E_MI_AI_IF_ADC_CD, &s8IfLeftGain, &s8IfRightGain), MI_SUCCESS);
4. ExecFunc(MI_AI_GetIfGain(E_MI_AI_IF_DMIC_A_01, &s8IfLeftGain, &s8IfRightGain), MI_SUCCESS);
5. ExecFunc(MI_AI_GetIfGain(E_MI_AI_IF_DMIC_A_23, &s8IfLeftGain, &s8IfRightGain), MI_SUCCESS);
```

2.15. MI_AI_SetIfMute

- 功能

设置音频输入外设的静音参数。
- 语法


```
MI_S32 MI_AI_SetIfMute(MI\_AI\_If\_e enAiIf, MI_BOOL bLeftMute, MI_BOOL bRightMute);
```

- 形参

参数名称	描述	输入/输出
<code>enAiIf</code>	音频输入外设	输入
<code>bLeftMute</code>	左声道静音参数	输入
<code>bRightMute</code>	右声道静音参数	输入

- 返回值

返回值

{

0 成功。

非 0 失败，参照[错误码](#)。

- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.so/libmi_ai.a`

- ※ 注意
 - 目前没有输入外设支持此设定。

- 举例


```
1. ExecFunc(MI_AI_SetIfMute(E_MI_AI_IF_ADC_AB, TRUE, FALSE), MI_SUCCESS);
2. ExecFunc(MI_AI_SetIfMute(E_MI_AI_IF_ADC_CD, TRUE, FALSE), MI_SUCCESS);
3. ExecFunc(MI_AI_SetIfMute(E_MI_AI_IF_DMIC_A_01, TRUE, FALSE), MI_SUCCESS);
4. ExecFunc(MI_AI_SetIfMute(E_MI_AI_IF_DMIC_A_23, TRUE, FALSE), MI_SUCCESS);
```

2.16. MI_AI_GetIfMute

➤ 功能

获取音频输入外设的静音参数。

➤ 语法

```
MI_S32 MI_AI_GetIfMute(MI_AI_If_e enAiIf, MI_BOOL *pbLeftMute, MI_BOOL *pbRightMute);
```

➤ 形参

参数名称	描述	输入/输出
enAiIf	音频输入外设	输入
pbLeftMute	左声道静音参数指针	输出
pbRightMute	右声道静音参数指针	输出

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.so/libmi_ai.a

※ 注意

无。

➤ 举例

```
1. MI_BOOL bIfLeftMute, bIfRightMute;
2. ExecFunc(MI_AI_GetIfMute(E_MI_AI_IF_ADC_AB, &bIfLeftMute, &bIfRightMute), MI_SUCCESS);
3. ExecFunc(MI_AI_GetIfMute(E_MI_AI_IF_ADC_CD, &bIfLeftMute, &bIfRightMute), MI_SUCCESS);
4. ExecFunc(MI_AI_GetIfMute(E_MI_AI_IF_DMIC_A_01, &bIfLeftMute, &bIfRightMute), MI_SUCCESS);
5. ExecFunc(MI_AI_GetIfMute(E_MI_AI_IF_DMIC_A_23, &bIfLeftMute, &bIfRightMute), MI_SUCCESS);
```

2.17. MI_AI_SetI2SConfig

➤ 功能

设置 I2S RX 的配置信息。

➤ 语法

```
MI_S32 MI_AI_SetI2SConfig(MI_AI_If_e enAiI2SIf, const MI_AUDIO_I2sConfig_t *pstConfig);
```

➤ 形参

参数名称	描述	输入/输出
enAiI2SIf	音频I2S RX输入外设	输入
pstConfig	音频I2S RX的配置信息	输入

➤ 返回值

返回值

0	成功。
---	-----

返回值

非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.so/libmi_ai.a`

※ 注意

- 一组 I2S RX 只需要配置一次。比如使用 `E_MI_AI_IF_I2S_A_01` 配置后，`I2S_A` 已被初始化，后面使用其他属于 `I2S_A` 的 interface，无需再次对 `I2S_A` 做配置，除非 `I2S_A` 的配置参数有改变。
- [MI_AUDIO_I2sConfig_t](#) 中的 `enSampleRate` 必须和 [MI_AI_Attr_t](#) 中的 `enSampleRate` 一致，否则 attach 会报错。

➤ 举例

```
1. MI_AUDIO_I2sConfig_t stAiI2sACfg;
2. memset(&stAiI2sACfg, 0x0, sizeof(stAiI2sACfg));
3. stAiI2sACfg.enMode = E_MI_AUDIO_I2S_MODE_I2S_MASTER;
4. stAiI2sACfg.enFormat = E_MI_AUDIO_I2S_FMT_I2S_MSB;
5. stAiI2sACfg.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
6. stAiI2sACfg.enMclk = E_MI_AUDIO_I2S_MCLK_0;
7. stAiI2sACfg.bSyncClock = TRUE;
8. stAiI2sACfg.u32TdmSlots = 2;
9. stAiI2sACfg.enBitWidth = E_MI_AUDIO_BIT_WIDTH_16;
10. ExecFunc(MI_AI_SetI2SConfig(E_MI_AI_IF_I2S_A_01, &stAiI2sACfg), MI_SUCCESS);
```

2.18. MI_AI_GetI2SConfig

➤ 功能

获取 I2S RX 的配置信息。

➤ 语法

`MI_S32 MI_AI_GetI2SConfig(MI\_AI\_If\_e enAiI2SIf, MI\_AUDIO\_I2sConfig\_t *pstConfig);`

➤ 形参

参数名称	描述	输入/输出
<code>enAiI2SIf</code>	音频 I2S RX 输入外设	输入
<code>pstConfig</code>	音频 I2S RX 的配置信息	输出

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.so/libmi_ai.a`

※ 注意

无。

➤ 举例

```
1. MI_AUDIO_I2sConfig_t stAiI2sACfg;
2. memset(&stAiI2sACfg, 0x0, sizeof(stAiI2sACfg));
3. ExecFunc(MI_AI_GetI2SConfig(E_MI_AI_IF_I2S_A_01, &stAiI2sACfg), MI_SUCCESS);
```

2.19. MI_AI_DupChnGroup

➤ 功能

同步音频输入通道组的状态。

➤ 语法

MI_S32 MI_AI_DupChnGroup([MI_AUDIO_DEV](#) AiDevId, MI_U8 u8ChnGrpIdx)

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
u8ChnGrpIdx	音频输入通道组号 范围为[0, Channel Group Number - 1] Channel Group Number = (attach上的物理通道数 - attach上的Echo通道数) / Sound Mode	输入

➤ 返回值

返回值

0	成功。
非0	失败，参照 错误码 。

➤ 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.so/libmi_ai.a`

※ 注意

- 此接口仅适用于 `dual os` 环境。在 `RTOS` 环境下已初始化音频输入通道组，切换到 `Linux` 环境时，用于同步音频输入通道组的状态。

➤ 举例

无。

2.20. MI_AI_InitDev

➤ 功能

初始化音频输入设备。

➤ 语法

MI_S32 MI_AI_InitDev([MI_AI_InitParam_t](#) *pstInitParam);

➤ 形参

参数名称	描述	输入/输出
------	----	-------

pstInitParam	设备初始化参数	输入
--------------	---------	----

- 返回值

返回值	{	0 成功。 非 0 失败，参照 错误码 。
-----	---	---
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a
- ※ 注意
 - 本接口必须和 [MI_AI_DeInitDev](#) 成对使用，不可单独重复调用，否则返回失败。
 - 仅用于 STR 状态唤醒后重新初始化 AI 模块。
- 举例

无。

2.21. MI_AI_DeInitDev

- 功能

反初始化音频输入设备。
- 语法


```
MI_S32 MI_AI_DeInitDev(void);
```
- 形参

参数名称	描述	输入/输出
- 返回值

返回值	{	0 成功。 非 0 失败，参照 错误码 。
-----	---	---
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a
- ※ 注意
 - 此函数必须在初始化设备后调用，否则返回失败。
 - 如果本接口在 app 退出前没有调用，内部会自动反初始化设备。
 - 本接口必须和 [MI_AI_InitDev](#) 成对使用，不可单独重复调用，否则返回失败。
- 举例

无。

2.22. MI_AI_GetAttr

➤ 功能

获取音频输入设备的属性。

➤ 语法

```
MI_S32 MI_AI_GetAttr(MI_AUDIO_DEV AiDevId, MI_AI_Attr_t *pstAttr);
```

➤ 形参

参数名称	描述	输入/输出
AiDevId	音频输入设备号	输入
pstAttr	音频输入设备属性	输出

➤ 返回值

返回值

{	0	成功。
	非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.so/libmi_ai.a

※ 注意

- 此函数必须在打开设备成功后调用，否则返回失败。

➤ 举例

无。

3. AI 数据类型

AI 模块相关数据类型定义如下：

数据类型	定义
MI_AUDIO_DEV	定义音频输入设备号
MI_AUDIO_Format_e	定义音频输入设备的数据格式
MI_AUDIO_SoundMode_e	定义音频输入设备的声音模式
MI_AUDIO_SampleRate_e	定义音频采样率
MI_AI_Attr_t	定义音频输入设备属性结构体
MI_AI_If_e	定义音频输入
MI_AI_MAX_CHN_NUM	定义音频输入设备支持的最大物理通道数
MI_AI_Data_t	定义音频数据结构体
MI_AUDIO_I2sMode_e	定义 I2S RX/TX 的工作模式
MI_AUDIO_I2sBitWidth_e	定义 I2S RX/TX 的位宽
MI_AUDIO_I2sFormat_e	定义 I2S RX/TX 的传输数据格式
MI_AUDIO_I2sMclk_e	定义 I2S RX/TX 的 mclk 时钟频率
MI_AUDIO_I2sConfig_t	定义 I2S RX/TX 的配置信息
MI_AI_InitParam_t	定义音频输入设备初始化参数

3.1. MI_AUDIO_DEV

- 说明
定义音频输入/输出设备号。
- 定义
typedef MI_S32 MI_AUDIO_DEV
- ※ 注意事项
无。
- 相关数据类型及接口
无。

3.2. MI_AUDIO_Format_e

- 说明
定义音频数据格式。
- 定义
typedef enum
{

```
E_MI_AUDIO_FORMAT_INVALID = -1,
E_MI_AUDIO_FORMAT_PCM_S16_LE = 0,
} MI_AUDIO_Format_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_FORMAT_INVALID	非法的数据格式
E_MI_AUDIO_FORMAT_PCM_S16_LE	PCM Linear 16bit (Little Endian)

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_AI_Attr_t](#)

3.3. MI_AUDIO_SoundMode_e

➤ 说明

定义音频声音模式。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_SOUND_MODE_MONO = 1,
    E_MI_AUDIO_SOUND_MODE_STEREO = 2,
    E_MI_AUDIO_SOUND_MODE_4CH = 4,
    E_MI_AUDIO_SOUND_MODE_6CH = 6,
    E_MI_AUDIO_SOUND_MODE_8CH = 8,
}MI_AUDIO_SoundMode_e
```

➤ 成员

成员名称	描述
E_MI_AUDIO_SOUND_MODE_MONO	单声道。
E_MI_AUDIO_SOUND_MODE_STEREO	双声道。
E_MI_AUDIO_SOUND_MODE_4CH	四声道。
E_MI_AUDIO_SOUND_MODE_6CH	六声道。
E_MI_AUDIO_SOUND_MODE_8CH	八声道。

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_AI_Attr_t](#)

3.4. MI_AUDIO_SampleRate_e

➤ 说明

定义音频采样率。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_SAMPLE_RATE_8000 = 8000,
    E_MI_AUDIO_SAMPLE_RATE_11025 = 11025,
    E_MI_AUDIO_SAMPLE_RATE_12000 = 12000,
    E_MI_AUDIO_SAMPLE_RATE_16000 = 16000,
    E_MI_AUDIO_SAMPLE_RATE_22050 = 22050,
    E_MI_AUDIO_SAMPLE_RATE_24000 = 24000,
    E_MI_AUDIO_SAMPLE_RATE_32000 = 32000,
    E_MI_AUDIO_SAMPLE_RATE_44100 = 44100,
    E_MI_AUDIO_SAMPLE_RATE_48000 = 48000,
    E_MI_AUDIO_SAMPLE_RATE_96000 = 96000,
    E_MI_AUDIO_SAMPLE_RATE_192000 = 192000,
}MI_AUDIO_SampleRate_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_SAMPLE_RATE_8000	8kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_11025	11.025kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_12000	12kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_16000	16kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_22050	22.05kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_24000	24kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_32000	32kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_44100	44.1kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_48000	48kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_96000	96kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_192000	192kHz 采样率

※ 注意事项

- 除 I2S RX, 其他的音频输入外设仅支持 8/16/32/48kHz。
- I2S RX 可支持 8/16/32/48/96/192kHz, 但使用 96/192kHz 只能单独 attach I2S RX。
- Muffin 芯片 I2S RX 仅支持 8/16/32/48/96 kHz。

➤ 相关数据类型及接口

[MI_AI_Attr_t](#)

3.5. MI_AI_Attr_t

➤ 说明

定义音频输入设备属性结构体。

➤ 定义

```
typedef struct MI_AUDIO_Attr_s
{
    MI\_AUDIO\_Format\_e enFormat;
    MI\_AUDIO\_SoundMode\_e enSoundMode;
    MI\_AUDIO\_SampleRate\_e enSampleRate;
```

```
MI_U32 u32PeriodSize;  
MI_BOOL bInterleaved;  
}MI_AI_Attr_t;
```

➤ 成员

成员名称	描述
enFormat	音频数据格式。 静态属性。
enSoundMode	音频声音模式。 静态属性。 声音模式决定一个通道组对应多少个物理通道。
enSampleRate	音频采样率。 静态属性。
u32PeriodSize	音频数据的样本数。 静态属性。
bInterleaved	音频数据是否为交错模式。 静态属性。

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_AI_Open](#)

3.6. MI_AI_If_e

➤ 说明
定义音频输入外设。

➤ 定义

```
typedef enum  
{  
    E_MI_AI_IF_NONE = 0,  
    E_MI_AI_IF_ADC_AB = 1,  
    E_MI_AI_IF_ADC_CD = 2,  
    E_MI_AI_IF_DMIC_A_01 = 3,  
    E_MI_AI_IF_DMIC_A_23 = 4,  
    E_MI_AI_IF_I2S_A_01 = 5,  
    E_MI_AI_IF_I2S_A_23 = 6,  
    E_MI_AI_IF_I2S_A_45 = 7,  
    E_MI_AI_IF_I2S_A_67 = 8,  
    E_MI_AI_IF_I2S_A_89 = 9,  
    E_MI_AI_IF_I2S_A_ab = 10,  
    E_MI_AI_IF_I2S_A_cd = 11,  
    E_MI_AI_IF_I2S_A_ef = 12,  
    E_MI_AI_IF_I2S_B_01 = 13,  
    E_MI_AI_IF_I2S_B_23 = 14,  
    E_MI_AI_IF_I2S_B_45 = 15,  
    E_MI_AI_IF_I2S_B_67 = 16,  
    E_MI_AI_IF_I2S_B_89 = 17,  
    E_MI_AI_IF_I2S_B_ab = 18,
```

```

E_MI_AI_IF_I2S_B_cd = 19,
E_MI_AI_IF_I2S_B_ef = 20,
E_MI_AI_IF_I2S_C_01 = 21,
E_MI_AI_IF_I2S_C_23 = 22,
E_MI_AI_IF_I2S_C_45 = 23,
E_MI_AI_IF_I2S_C_67 = 24,
E_MI_AI_IF_I2S_C_89 = 25,
E_MI_AI_IF_I2S_C_ab = 26,
E_MI_AI_IF_I2S_C_cd = 27,
E_MI_AI_IF_I2S_C_ef = 28,
E_MI_AI_IF_I2S_D_01 = 29,
E_MI_AI_IF_I2S_D_23 = 30,
E_MI_AI_IF_I2S_D_45 = 31,
E_MI_AI_IF_I2S_D_67 = 32,
E_MI_AI_IF_I2S_D_89 = 33,
E_MI_AI_IF_I2S_D_ab = 34,
E_MI_AI_IF_I2S_D_cd = 35,
E_MI_AI_IF_I2S_D_ef = 36,
E_MI_AI_IF_ECHO_A = 37,
E_MI_AI_IF_HDMI_A = 38,
E_MI_AI_IF_DMIC_A_45 = 39,
E_MI_AI_IF_MAX,
} MI_AI_If_e;

```

➤ 成员

成员名称	描述
E_MI_AI_IF_NONE	None
E_MI_AI_IF_ADC_AB	ADC0/1
E_MI_AI_IF_ADC_CD	ADC2/3
E_MI_AI_IF_DMIC_A_01	DMIC Chn0 和 Chn1
E_MI_AI_IF_DMIC_A_23	DMIC Chn2 和 Chn3
E_MI_AI_IF_I2S_A_01	I2S RX A 的 Chn0 和 Chn1
E_MI_AI_IF_I2S_A_23	I2S RX A 的 Chn2 和 Chn3
E_MI_AI_IF_I2S_A_45	I2S RX A 的 Chn4 和 Chn5
E_MI_AI_IF_I2S_A_67	I2S RX A 的 Chn6 和 Chn7
E_MI_AI_IF_I2S_A_89	I2S RX A 的 Chn8 和 Chn9
E_MI_AI_IF_I2S_A_ab	I2S RX A 的 Chn10 和 Chn11
E_MI_AI_IF_I2S_A_cd	I2S RX A 的 Chn12 和 Chn13
E_MI_AI_IF_I2S_A_ef	I2S RX A 的 Chn14 和 Chn15
E_MI_AI_IF_I2S_B_01	I2S RX B 的 Chn0 和 Chn1
E_MI_AI_IF_I2S_B_23	I2S RX B 的 Chn2 和 Chn3
E_MI_AI_IF_I2S_B_45	I2S RX B 的 Chn4 和 Chn5
E_MI_AI_IF_I2S_B_67	I2S RX B 的 Chn6 和 Chn7
E_MI_AI_IF_I2S_B_89	I2S RX B 的 Chn8 和 Chn9
E_MI_AI_IF_I2S_B_ab	I2S RX B 的 Chn10 和 Chn11
E_MI_AI_IF_I2S_B_cd	I2S RX B 的 Chn12 和 Chn13
E_MI_AI_IF_I2S_B_ef	I2S RX B 的 Chn14 和 Chn15
E_MI_AI_IF_I2S_C_01	I2S RX C 的 Chn0 和 Chn1
E_MI_AI_IF_I2S_C_23	I2S RX C 的 Chn2 和 Chn3
E_MI_AI_IF_I2S_C_45	I2S RX C 的 Chn4 和 Chn5
E_MI_AI_IF_I2S_C_67	I2S RX C 的 Chn6 和 Chn7
E_MI_AI_IF_I2S_C_89	I2S RX C 的 Chn8 和 Chn9

成员名称	描述
E_MI_AI_IF_I2S_C_ab	I2S RX C 的 Chn10 和 Chn11
E_MI_AI_IF_I2S_C_cd	I2S RX C 的 Chn12 和 Chn13
E_MI_AI_IF_I2S_C_ef	I2S RX C 的 Chn14 和 Chn15
E_MI_AI_IF_I2S_D_01	I2S RX D 的 Chn0 和 Chn1
E_MI_AI_IF_I2S_D_23	I2S RX D 的 Chn2 和 Chn3
E_MI_AI_IF_I2S_D_45	I2S RX D 的 Chn4 和 Chn5
E_MI_AI_IF_I2S_D_67	I2S RX D 的 Chn6 和 Chn7
E_MI_AI_IF_I2S_D_89	I2S RX D 的 Chn8 和 Chn9
E_MI_AI_IF_I2S_D_ab	I2S RX D 的 Chn10 和 Chn11
E_MI_AI_IF_I2S_D_cd	I2S RX D 的 Chn12 和 Chn13
E_MI_AI_IF_I2S_D_ef	I2S RX D 的 Chn14 和 Chn15
E_MI_AI_IF_ECHO_A	SRC 数据（回声）
E_MI_AI_IF_HDMI_A	HDMI RX
E_MI_AI_IF_DMIC_A_45	DMIC Chn4 和 Chn5

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_AI_AttachIf](#)

[MI_AI_SetIfGain](#)

[MI_AI_GetIfGain](#)

[MI_AI_SetIfMute](#)

[MI_AI_GetIfMute](#)

[MI_AI_SetI2SConfig](#)

[MI_AI_GetI2SConfig](#)

3.7. MI_AI_MAX_CHN_NUM

➤ 说明

定义音频输入设备支持的最大物理通道数。

➤ 定义

```
#define MI_AI_MAX_CHN_NUM 8
```

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.8. MI_AI_Data_t

➤ 说明

定义音频结构体。

➤ 定义

```
typedef struct MI_AI_Data_s
```



```
{
    void *apvBuffer[MI_AI_MAX_CHN_NUM];
    MI_U32 u32Bytes[MI_AI_MAX_CHN_NUM];
    MI_U64 u64Pts;
    MI_U64 u64Seq;
}MI_AI_Data_t;
```

➤ 成员

成员名称	描述
apvBuffer	音频数据地址。 若音频输入设备使能交错模式，则数据保存在 apvBuffer[0] 中。 若音频输入设备未使能交错模式，则数据按顺序排列在 apvBuffer 中。
u32Bytes	音频数据长度。 与 apvBuffer 一一对应。
u64Pts	音频数据时间戳。 以 μ s 为单位。
u64Seq	音频数据序号。

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_AI_Read](#)

[MI_AI_ReleaseData](#)

3.9. MI_AUDIO_I2sMode_e

➤ 说明

定义音频 I2S RX/TX 的工作模式。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_I2S_MODE_I2S_MASTER,
    E_MI_AUDIO_I2S_MODE_I2S_SLAVE,
    E_MI_AUDIO_I2S_MODE_TDM_MASTER,
    E_MI_AUDIO_I2S_MODE_TDM_SLAVE,
} MI_AUDIO_I2sMode_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_I2S_MODE_I2S_MASTER	I2S 主模式
E_MI_AUDIO_I2S_MODE_I2S_SLAVE	I2S 从模式
E_MI_AUDIO_I2S_MODE_TDM_MASTER	TDM 主模式
E_MI_AUDIO_I2S_MODE_TDM_SLAVE	TDM 从模式

※ 注意事项

- 主模式与从模式是否支持会依据不同的芯片而有区别。

- 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#)

3.10. MI_AUDIO_I2sBitWidth_e

- 说明

定义音频 I2S RX/TX 的位宽。

- 定义

```
typedef enum
{
    E_MI_AUDIO_BIT_WIDTH_16,
    E_MI_AUDIO_BIT_WIDTH_32,
} MI_AUDIO_I2sBitWidth_e;
```

- 成员

成员名称	描述
E_MI_AUDIO_BIT_WIDTH_16	I2S 位宽为 16bit。
E_MI_AUDIO_BIT_WIDTH_32	I2S 位宽为 32bit。

- ※ 注意事项
无。

- 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#)

3.11. MI_AUDIO_I2sFormat_e

- 说明

定义音频 I2S RX/TX 的数据传输格式。

- 定义

```
typedef enum
{
    E_MI_AUDIO_I2S_FMT_I2S_MSB,
    E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB,
} MI_AUDIO_I2sFormat_e;
```

- 成员

成员名称	描述
E_MI_AUDIO_I2S_FMT_I2S_MSB	I2S 标准格式，最高位优先
E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB	I2S 左对齐格式，最高位优先

- ※ 注意事项
无。

- 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#)

3.12. MI_AUDIO_I2sMclk_e

➤ 说明

定义音频 I2S RX/TX 的 mclk 时钟频率。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_I2S_MCLK_0,
    E_MI_AUDIO_I2S_MCLK_12_288M,
    E_MI_AUDIO_I2S_MCLK_16_384M,
    E_MI_AUDIO_I2S_MCLK_18_432M,
    E_MI_AUDIO_I2S_MCLK_24_576M,
    E_MI_AUDIO_I2S_MCLK_24M,
    E_MI_AUDIO_I2S_MCLK_48M,
} MI_AUDIO_I2sMclk_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_I2S_MCLK_0	关闭 MCLK
E_MI_AUDIO_I2S_MCLK_12_288M	设置 MCLK 为 12.88M
E_MI_AUDIO_I2S_MCLK_16_384M	设置 MCLK 为 16.384M
E_MI_AUDIO_I2S_MCLK_18_432M	设置 MCLK 为 18.432M
E_MI_AUDIO_I2S_MCLK_24_576M	设置 MCLK 为 24.576M
E_MI_AUDIO_I2S_MCLK_24M	设置 MCLK 为 24M
E_MI_AUDIO_I2S_MCLK_48M	设置 MCLK 为 48M

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#)

3.13. MI_AUDIO_I2sConfig_t

➤ 说明

定义 I2S RX/TX 的配置信息。

➤ 定义

```
typedef struct MI_AUDIO_I2sConfig_s
{
    MI_AUDIO_I2sMode_e enMode;
    MI_AUDIO_I2sBitWidth_e enBitWidth;
    MI_AUDIO_I2sFormat_e enFormat;
    MI_AUDIO_SampleRate_e enSampleRate;
    MI_AUDIO_I2sMclk_e enMclk;
```

```
MI_BOOL bSyncClock;
MI_U32 u32TdmSlots;
} MI_AUDIO_I2sConfig_t;
```

➤ 成员

成员名称	描述
enMode	I2S 的工作模式。
enBitWidth	I2S 的数据位宽。
enFormat	I2S 的传输数据格式。
enSampleRate	I2S 的采样率。
enMclk	I2S 的 mclk 时钟频率。
bSyncClock	I2S RX 和 I2S TX 是否共用时钟。
u32TdmSlots	I2S TDM slot 的数目（仅 TDM 模式下有效）

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_AI_SetI2SConfig](#)

[MI_AI_GetI2SConfig](#)

3.14. MI_AI_InitParam_t

➤ 说明

音频输入设备初始化参数。

➤ 定义

```
typedef struct MI_AI_InitParam_s
{
    MI_AUDIO_DEV AiDevId;
    MI_U8 *u8Data;
} MI_AI_InitParam_t;
```

➤ 成员

成员名称	描述
AiDevId	音频输入设备号
u8Data	参数指针（保留）

※ 注意事项

无。

➤ 数据类型及接口

[MI_AI_InitDev](#)

4. 错误码

AI API 错误码如下表所示:

错误码	宏定义	描述
0xA0042001	MI_AI_ERR_INVALID_DEVID	音频输入设备号无效
0xA0042002	MI_AI_ERR_INVALID_CHNGRPID	音频输入通道组号无效
0xA0042003	MI_AI_ERR_ILLEGAL_PARAM	音频输入参数设置无效
0xA0042006	MI_AI_ERR_NULL_PTR	输入参数空指标错误
0xA0042007	MI_AI_ERR_NOT_CONFIG	音频输入设备属性未设置
0xA0042008	MI_AI_ERR_NOT_SUPPORT	操作不支持
0xA0042009	MI_AI_ERR_NOT_PERM	操作不允许
0xA004200C	MI_AI_ERR_NOMEM	分配内存失败或相应的数据队列已满
0xA004200D	MI_AI_ERR_NOBUF	音频输入缓存不足
0xA004200E	MI_AI_ERR_BUF_EMPTY	音频输入缓存为空
0xA004200F	MI_AI_ERR_BUF_FULL	音频输入缓存为满
0xA0042010	MI_AI_ERR_SYS_NOTREADY	音频输入系统未初始化
0xA0042012	MI_AI_ERR_BUSY	音频输入系统忙碌
0xA0042017	MI_AI_ERR_NOT_ENABLED	音频输入设备或通道没有使能

5. PROCFS 介绍

5.1. cat

➤ 调试信息

```
# cat proc/mi_modules/mi_ai/mi_ai0
===== MI AI Dev0 Info =====

----- Start AI Dev0 Attr -----
DevStatus   Format   SoundMode   SampleRate   PeriodSize   Interleaved
started     S16_LE   Stereo      8000         1024         1

DmaBufSize   DmaBusySize   DmaFreeSize   TmpBufSize
65536        1088          64448         4096

Mha1ChnCnt   MiChnGrpCnt   ReadCountFromMha1
2            1             63

If slot[0]: ADC_AB
ADC_AB      :      Gain(0 ,0 ) Mute(0,0).

----- End AI Dev0 Attr -----

----- Start AI ChnGrp0 info -----
ChnGrpId    bEnable    ReadFrameCount
0           1          63
Mute Status:
Mute[0]: 0
Mute[1]: 0

Dpga Gain:
Dpga Gain[0]: 0
Dpga Gain[1]: 0

----- End AI ChnGrp0 info -----
```

➤ 调试信息分析

记录当前 AI 的使用状况以及 device 属性、channel group 属性，可以动态地获取到这些信息，方便调试和测试。

➤ 参数说明

参数		描述
AI Device	DevStatus	AI 设备状态

参数		描述
Attr (AI 设备属性)		uninit: 未初始化 opened: open 成功
	Format	音频格式 (位宽及大小端等) 目前仅支持 S16_LE (16bit, 小端模式)
	SoundMode	声音模式 mono: 单声道 stereo: 立体声 SOUND_MODE_4CH: 4channel SOUND_MODE_6CH: 6channel SOUND_MODE_8CH: 8channel
	SampleRate	采样率 8k/16k/32k/48k
	PeriodSize	AI 每帧数据量
	Interleaved	是否为交错模式
	DmaBufSize	DMA buffer 的大小
	DmaBusySize	DMA buffer 已使用的大小
	DmaFreeSize	DMA buffer 未使用的大小
	MhalChnCnt	mhal 设备的通道数
	MiChnGrpCnt	MI channel group 数
	ReadCountFromMhal	AI 从 mhal 读取的帧数
	If slot[IfIdx]: interface type	[IfIdx]: Interface 索引 interface type: Interface 类型
	[interface type]: Gain(Leftgain, RightGain) Mute(LeftMute, RightMute)	[interface type]: Interface 类型 Gain(Leftgain, RightGain): 左声道增益, 右声道增益 Mute(LeftMute, RightMute): 左声道静音, 右声道静音
AI I2S Status (AI I2S 信息)	I2sMode	I2S Rx 的工作模式(仅 interface 为 I2S Rx 时有效) i2s-master i2s-slave tdm-master tdm-slave
	I2sMclk	I2S Rx 的 Mclk 的频率(仅 interface 为 I2S Rx 时有效) disable: 不使用 Mclk 其他值为当前的 Mclk 频率
	I2sFmt	I2S Rx 的数据格式(仅 interface 为 I2S Rx 时有效)

参数		描述
		I2S-MSB: I2S 格式 LEFT-MSB: I2S 左对齐格式
	bI2sSync	I2S RX 和 TX 是否共用 clock(仅 interface 为 I2S Rx 时有效) 1: 4 wire mode, RX 和 TX 共用 clock 0: 6 wire mode, RX 和 TX 都有独立的 clock
	TdmSlots	I2S Rx 的 TDM slot 数目(仅 interface 为 I2S Rx 为 TDM 模式时有效)
	I2sBitWidth	I2S RX 的位宽(仅 interface 为 I2S Rx, 且需要支持 TDM 模式的芯片有效)
AI ChnGrp info (AI 通道 信息)	ChnGrpId	MI channel group ID
	bEnable	是否使能
	ReadFrameCount	对应 channel group 读取的帧数
	Mute Status	DPGA Mute 信息
	Mute[PhyChnIdx]:bMute	[PhyChnIdx]:channel group 中的 channel 索引 bMute: 静音状态
	Dpga Gain	DPGA 增益信息
	Dpga Gain[PhyChnIdx]:Gain	[PhyChnIdx]: channel group 中的 channel 索引 Gain:增益

5.2. echo

功能	动态启用/关闭 AI 设备 dump mhal pcm 数据
命令	echo dump_mhal_chn [Path] [ON/on/1, OFF/off/0] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[Path] dump 文件保存的路径
	[ON/on/1, OFF/off/0] 是否使能
	[ID] AI 设备号
举例	echo dump_mhal_chn /tmp 1 > proc/mi_modules/mi_ao/mi_ao0

功能	动态启用/关闭 AI 设备 dump mi pcm 数据
命令	echo dump_mi_chn [ChnGrpId] [Path] [ON/on/1, OFF/off/0] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[ChnGrpId] channel group ID
	[Path] dump 文件保存的路径
	[ON/on/1, OFF/off/0] 是否使能
	[ID] AI 设备号

举例	echo dump_mi_chn 0 /tmp 1 > proc/mi_modules/mi_ao/mi_ai0
----	--

功能	动态设定 AI DPGA 的增益
命令	echo set_dpga_gain [ChnGrpId] [ChnId] [Gain] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[ChnGrpId] channel group ID
	[ChnId] channel group 中的 channel 索引
	[Gain] AI 数字增益
举例	echo set_dpga_gain 0 0 -10 > proc/mi_modules/mi_ao/mi_ai0

功能	动态设定 AI Interface 的增益
命令	echo set_inf_gain [If] [LeftGain] [RightGain] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[If] AI Interface
	[LeftGain] [RightGain] 左声道增益，右声道增益
举例	echo set_inf_gain 1 18 18 > proc/mi_modules/mi_ao/mi_ai0

功能	动态设定 AI DPGA 静音模式
命令	echo set_dpga_mute [ChnGrpId] [ChnId] [Mute] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[ChnGrpId] channel group ID
	[ChnId] channel group 中的 channel 索引
	[Mute] AI DPGA 静音
举例	echo set_dpga_mute 0 0 1 > proc/mi_modules/mi_ao/mi_ai0

功能	动态设定 AI Interface 静音模式
命令	echo set_inf_mute [If] [LeftMute] [RightMute] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[If] AI Interface
	[LeftMute] [RightMute] 左声道静音，右声道静音
举例	echo set_inf_mute 1 0 1 > proc/mi_modules/mi_ao/mi_ai0

功能	动态开启/关闭 AI 设备的 Singen 功能
命令	echo singen [SinGen Index] [Enable] > proc/mi_modules/mi_ai/mi_ai[ID]
参数说明	[SinGen Index] 需要控制的 Singen ID
	[Enable] 开启/关闭 Singen 功能
举例	echo singen 0 1 > proc/mi_modules/mi_ao/mi_ai0